



VERSION CONTROL / GIT WORKFLOWS

Version Controlling with Git - II

Branching, pull requests, peer review and GitHub Flow

Practical focus

This lab continues the Git basics practical. You will isolate work in a feature branch, publish the branch, create and review pull requests, and complete a simple GitHub Flow cycle individually and with a colleague.

Learning outcomes

- Create and switch between branches using a clear naming convention.
- Push a local feature branch and set its remote upstream branch.
- Create, review, revise and merge a pull request.
- Collaborate safely through a shared GitHub repository.
- Apply GitHub Flow and present evidence of the completed workflow.

Prerequisites

- Git is installed and configured with your name and email.
- You can sign in to GitHub and create a repository.
- You have completed the previous Git basics practical.
- You have a colleague available for the pair-work activities.

Lab convention

Use main as the default branch. Name feature branches as feature/<student-id>/<short-feature-name>. Keep commits small and use messages that describe the completed change.



VERSION CONTROL / GIT WORKFLOWS

Part 1 - Individual Work

Create a feature branch and complete a pull request

- 1 Create a new public GitHub repository. Add a README if desired.
- 2 Clone the repository to your computer and open the cloned directory in a terminal.
- 3 Create and immediately switch to a feature branch. Use a meaningful name that follows the lab convention.

```
git clone <repository-url>
cd <repository-folder>
git switch -c feature/<student-id>/profile-page
```

Alternative branch commands

git branch <branch-name> creates a branch only; use git switch <branch-name> to move to it. git checkout -b <branch-name> is the older one-command alternative.

- 4 Add one or more small text, HTML or source files. The content can be simple, but the changes must be easy to identify.
- 5 Inspect the changes, stage them, create an atomic commit and push the branch to GitHub.

```
git status
git add .
git commit -m "Add profile page structure"
git push -u origin feature/<student-id>/profile-page
```

- 6 Verify that the feature branch and its files are visible on GitHub.
- 7 Create a pull request with main as the base branch. Use a concise title and describe what changed and how it was checked.
- 8 Review the changed files, merge the pull request and delete the remote feature branch.
- 9 Return to the local main branch, pull the merged changes and confirm the working tree is clean.

```
git switch main
git pull origin main
git status
```

VERSION CONTROL / GIT WORKFLOWS

Part 2 - Pair Collaboration

Contribute to a colleague's repository through review

Pair-work rule

Do not share passwords, access tokens or personal credentials. Collaborate through GitHub invitations, branches and pull requests.

- 1 Pair with a colleague. Student A invites Student B as a collaborator using the colleague's GitHub username.
- 2 Student B accepts the invitation and clones Student A's repository into a separate local directory.
- 3 Student B creates a feature branch using the required naming format, makes a small change and commits it.
- 4 Student B pushes the branch and creates a pull request to Student A's main branch.
- 5 Student A reviews the pull request and leaves at least one useful review comment or approval.
- 6 If changes are requested, Student B updates the same branch and pushes again. The pull request updates automatically.
- 7 Student A merges the accepted pull request and deletes the remote feature branch.
- 8 Reverse the roles so that both students complete the contributor and reviewer responsibilities.

Pull request review checklist

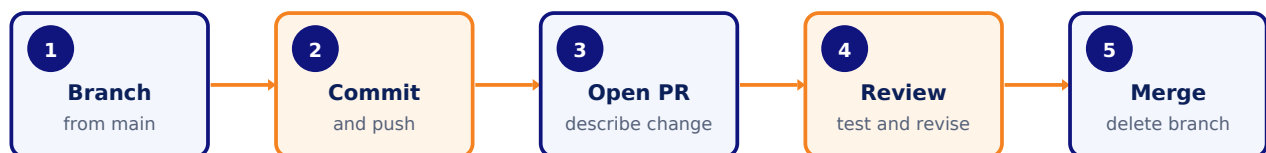
Review point	What to check
Scope	The pull request contains only the intended feature changes.
Clarity	Files, branch, commits and pull request title are meaningful.
Correctness	The changed files open or run as expected and introduce no obvious issue.
Discussion	Feedback is specific, respectful and actionable.

VERSION CONTROL / GIT WORKFLOWS

Part 3 - GitHub Flow

Apply a lightweight branch, review and merge workflow

GitHub Flow is a simple workflow for short-lived feature branches. Work begins from an up-to-date main branch, changes are discussed and checked through a pull request, and accepted work is merged back into main.



- 1 Update your local main branch before starting new work.
- 2 Create a clearly named feature branch from main.
- 3 Make several small, related commits and push the branch regularly.
- 4 Open a pull request and ask your colleague to review it.
- 5 Respond to feedback by updating the same branch; test the final result.
- 6 After approval, merge the pull request and delete the feature branch.
- 7 Update local main and remove the completed local branch.

```
git switch main
git pull origin main
git branch -d feature/<student-id>/<short-feature-name>
git fetch --prune
git log --oneline --graph --decorate --all
```

Official references

- Git Reference - official Git documentation
- git switch - official command reference
- Pull requests - GitHub Docs
- Inviting collaborators - GitHub Docs
- GitHub Flow walkthrough - GitHub Docs

VERSION CONTROL / GIT WORKFLOWS

Part 4 - Lab Submission

Submit workflow evidence and answers to the following questions

Submission format

Submit one PDF containing the evidence checklist followed by your answers. Student-specific evidence is worth 5 marks and the written questions are worth 20 marks. Total: 25 marks.

A. Evidence checklist - 5 marks

Item	Evidence to submit
Repository	URL of your public GitHub repository.
Feature branch	Branch name and a screenshot showing it on GitHub.
Pull request	URL of the merged pull request and the reviewer's GitHub username.
Commit history	Output or screenshot of <code>git log --oneline --graph --decorate --all</code> .
Pair contribution	URL of the pull request you contributed to or reviewed in your colleague's repository.

B. Questions - 20 marks

1. Explain the difference between creating a branch with `git branch` and creating one with `git switch -c`. [2 marks]
2. Why should a new feature be developed in a feature branch instead of directly in main? [3 marks]
3. What is achieved by `git push -u origin <branch-name>`? [2 marks]
4. What is the purpose of a pull request, and what should a reviewer check before approving it? [3 marks]
5. Describe the GitHub Flow sequence from starting a feature to completing the work. [4 marks]
6. Why are meaningful branch names and small atomic commits important in team development? [2 marks]
7. Why is it good practice to delete a feature branch after it has been merged? [2 marks]
8. Your pull request has a conflict with main. State the steps you would take to resolve it safely and update the pull request. [2 marks]

Submission declaration

I confirm that the repository evidence and answers submitted are based on the practical work completed by me and my identified colleague.